GLOW: Accelerating Geometric Multigrid Levels on Cerebras Wafer-Scale Engine

Anonymous Submission

***Abstract*—This paper describes the mapping of Geometric Multigrid (GMG) onto the Cerebras Wafer Scale Engine (WSE-3). The WSE wafer integrates almost a million cores along with a low-latency 2D mesh network, providing unprecedented on-chip bandwidth and fine-grained communication. Although the architecture is predominantly used for deep learning, we explore the suitability of WSE for accelerating GMG. GMG solves sparse linear systems efficiently by recursively coarsening the grid, which leads to more rapid convergence with less computation. As compared to prior implementations of stencils on WSE, this demonstration highlights the challenges of mapping a layered algorithm – and the full set of operators – onto a dataflow architecture. We use this implementation as a foundation to discuss the potential for a variety of data and computation mappings of GMG to WSE, and performance compared to the HPGMG benchmark on an NVIDIA H200 GPU. Across different GMG configurations and problem sizes, we observe a speedup of up to 25× as compared to a single H200 node.**

***Index Terms*—Spatial dataflow architecture, Geometric multigrid, Wafer-Scale Engine**

I. Introduction

Geometric multigrid (GMG) is an important family of algorithms used by computational scientists to accelerate the convergence of iterative solvers for linear systems of the form $Ax = b$. It is a multigrid because computation is mapped to a hierarchy of coarser grid levels. As shown in Figure 1, the underlying GMG operators are organized in a V-cycle that includes stencil application, residual, restriction, interpolation, smooth, and a direct (bottom) solver. In such a formulation, the floating-point computation in GMG is dwarfed by the overhead of data movement (during stencil application and across levels), making managing the memory hierarchy and cross-processor communication critical to achieving high performance. The best-performing algorithm for multigrid solutions has always been driven by the architecture: for decades the focus has been on distributed-memory CPU clusters, and, more recently, by accelerators such as GPUs. While these systems deliver high peak floating-point throughput, the performance of the dominant smooth and bottom solver in multigrid is often memory-bound, limited by cache hierarchies, off-chip DRAM bandwidth, and the cost of communicating across nodes over high-latency interconnects.

This paper presents evaluates the implications of mapping GMG to the Cerebras Wafer-Scale Engine (WSE), a radically different architectural model [**?**], [**?**]. By integrating nearly one million processing elements (PEs) and tens of gigabytes of on-wafer SRAM onto a single wafer-scale chip, the WSE provides single-cycle access to local memory and 21 petabytes-per-second of memory bandwidth, along with 214 petabits-per-second of fabric bandwidth through a 2D mesh interconnect. As a result, algorithms that are typically memory-bound on CPUs and GPUs frequently become compute-bound when ported to the WSE. Prior work has demonstrated this shift for diverse workloads such as stencil computations [**?**], [**?**], [**?**], [**?**], [**?**], seismic analysis [**?**], molecular dynamics [**?**], [**?**], and Monte Carlo particle transport kernels [**?**], [**?**].

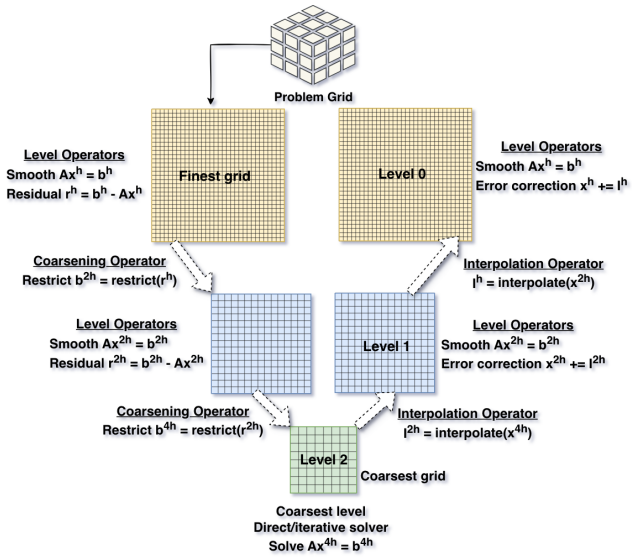

Fig. 1. Schematic representation of a Geometric Multigrid V-cycle illustrating smooth, restriction, coarse-grid correction, and interpolation operators. Small 3D grid represents the problem domain

In this paper, we present GLOW (Gmg Levels On Wafer). We build on the lessons from successful use of Cerebras for stencil computations to provide a mapping of an algorithm that is still structured yet significantly more challenging.

Our paper is centered around an iso-capability (same problem size) study of the WSE (44GB of SRAM) and a GH200 GPU (141GB of HBM DRAM). We present a dispassionate performance analysis of the WSE-3 focusing on the relevant benefits and pitfalls associated with massive decoupled thread parallelism, and planar network topologies for multi-scale computations. As compared to a GH200 GPU implementation of the HPGMG benchmark, we achieve significantly improved performance. At the same time, the implementation of GMG must address new challenges on the Cerebras architecture: (1) limitations on problem size that fit on the wafer; (2) relative communication costs growing at deeper levels of the V-Cycle; and, (3) trading off code size and data space in the memory-

constrained tile.

This paper makes the following contributions:

- The first implementation of a multigrid algorithm on Cerebras WS-3, to the best of our knowledge.
- A performance model of communication costs, along with a forwarding communication optimization, using the strided distributed solution we propose.
- Extensive performance measurements, including per-operator and per-level cost, along with a comparison with an NVIDIA GH200 running HPGMG that exhibits speedup of up to 25×.
- Lessons regarding GMG algorithm configuration suitable for a tiled architecture with limited memory.

Oscar points out we can talk about the software implementation (3.3?). Mary plans to draft a Lessons Learned section.

II. Background

This section summarizes the GMG approach and the key aspects of the Cerebras architecture relevant to this study.

A. Geometric Multigrid Methods

Multigrid methods are iterative solvers which solve sparse linear system of equations by forming a hierarchy of grid levels and iterating across them to improve accuracy. Geometric Multigrid (GMG) specifically uses the geometry of the problem to construct different levels and the grid hierarchy, as illustrated in Figure 1. In the figure, a three-level V-cycle is used to solve ($Ax = b$), where (A) is the stencil operator, (x) is the solution, (b) is the right-hand side, and superscripts (h), (2h), and (4h) correspond to grid spacings. Each level performs two primary operations: smoothing and residual evaluation. The smoother reduces high-frequency error, commonly with the weighted Jacobi smoother. The residual operator measures the deviation of the current iterate from the exact solution. The coarsening step restricts the residual to the next level, generating the coarse-grid right-hand side.

In finite-volume discretizations, restriction corresponds to averaging the contributions of eight fine-grid cells into one coarse cell, recursively repeated until reaching the coarsest grid, where the system is sufficiently small to be solved with a direct or iterative method such as conjugate gradient, BiCGSTAB or Jacobi. After the coarse solution is computed, the V-cycle traverses back up through the hierarchy. Interpolation transfers the coarse-grid correction to the finer grid such that a single coarse cell is replicated to 8 fine cells, the inverse of restriction. An error-correction step updates the fine-grid solution. Finally post smoothing reduces any high-frequency errors reintroduced during interpolation. This down-and-up traversal is called a V-cycle, and achieves rapid convergence. The cost of each V-cycle depends on the number of levels, the choice of operators and coarse-grid solver, the number of iterations on the coarse solver, and the performance of the stencil computation. Optimizing GMG therefore requires reducing the time spent in each operator and the interactions between levels.

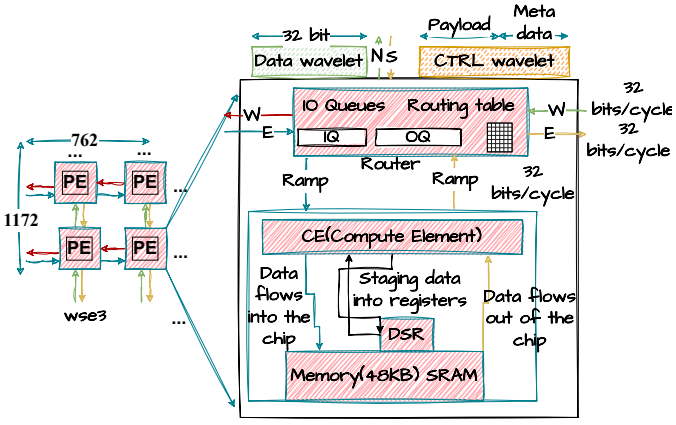

Fig. 2. A schematic representation of WSE-3, a 2D grid structure where each die is made of processing elements (figure left). Figure right shows the hardware machinery inside a PE: router, memory and Compute Element (CE). The WSE-3 system is a giant 46,255 mm² chip along with the machinery needed to cool, power and provide I/O.

B. Cerebras WSE-3 and Programming Model

Figure 2 shows the schematic representation of Cerebras WSE-3 Wafer-Scale Engine. The wafer consists of 893,064 identical, simple Processing Elements (PEs) arranged in a homogeneous 1172×762 2-dimensional grid. Each PE is self-contained with its own Memory, Compute Element (CE) and Router; i.e., the WSE is a spatial, distributed-memory, and dataflow-oriented architecture.

1) Router: A 5-port router moves the data across neighboring PEs (East, West, North, South links) and the CE (Ramp link) using 32-bit *wavelets*, the fundamental unit of communication. Routing is defined by 24 static virtual channels, called *colors*. In Figure 2, the colors red, blue, green and yellow show the routing of data into and out of the chip from neighboring directions (N,S,E, and W). A wavelet is assigned to a color, which encodes a routing path across PEs. A router decides, based on the wavelet's color and its configuration, where to send the wavelet. Multicasting replicates the wavelet and sends it in all directions with zero cost overhead. The router also has Queues for input output buffering.

For dynamic routing, the system provides *control wavelets*, which include additional metadata that allows routers to update entries in their routing tables at runtime and to trigger control tasks. All links operate at 32 bits per cycle.

2) CE and Memory: The CE executes integer and floating-point operations and includes FP16 and FP32 SIMD instructions that enable fast access to local memory. The CE consists of one main thread and up to eight microthreads, each of which may perform computation or handle I/O wavelets; in this way, computation and communication are overlapped. A small set of Data Structure Registers (DSRs) provide lightweight staging of data to/from the local 48KB SRAM, as shown in the figure. No data cache is needed since memory accesses exhibit single-cycle latency with 64-bit writes/cycle and 128-bit reads/cycle [**?**]. In total, the wafer has 44 GBytes of on-chip SRAM memory.

3) Programming Model: The programming interface, known as the Cerebras Software Language (CSL), exposes typical constructs including loops, functions, and conditionals, as well as low-level constructs such as Data DSDs, colors for routing, asynchronous tasks, and runtime router configuration [**?**]. Typically, the host code is written in Python where the programmer manages the initial data copy onto the spatial tiles along with allocation of tensors. CSL kernels manage the allocation of data on each PE and the corresponding computations. CSL layout code defines colors and routing across PEs; it also maps the kernel code spatially across the million PEs.

A CSL kernel consists of *functions* and *tasks*; data and control tasks describe dataflow, and execute only after being activated. Activation occurs when a wavelet tagged with a particular virtual channel (color) arrives at the PE's router, which can further trigger a change in router configurations or perform computations on the data. There are also local tasks that must be explicitly activated. This wavelet-triggered task model offers asynchronous fine-grained parallelism, but requires the programmer to manage communication and synchronization.

III. Mapping GMG to WSE-3

This section describes our mapping of GMG onto the WSE-3 wafer. We first consider the pros and cons of alternative approaches for mapping the problem grid and multi-grid levels onto the PEs in Section III-B. We describe the layout of the multi-grid data in Section III-C. Task activation is managed with a State Machine, as described in Section III-D. The design of key operators and a model of their communication are presented in Section III-E. We conclude with Section III-F by describing a forwarding optimization that optimizes the communication of data by bypassing the memory of nodes that do not participate in the communication.

A. Alternate Section 3.1: Computation Placement

I propose we use only text to describe alternatives that we found suboptimal, and eliminate all but the bottom row of Figure 3 and also eliminate Table 1 (Table 1 is now enclosed in comment environment). Here is my attempt at a shortened text. Mapping different multigrid levels onto a spatial accelerator with thousands of independent PEs introduces an opportunity for spatial placement of operators and levels across the wafer. We considered a number of alternative designs, some placing different operators in different areas of the wafer, and others spatially mapping the levels of the V-cycle. We briefly describe the alternatives here.

a) Pipelined Mapping of Operators:: In an operator-pipelined layout, one can place operators along the vertical dimension next to each other. At a finer granularity, the multigrid levels could also be pipelined. A pipelined spatial mapping is useful for data parallel and feedforward kinds of workloads. For example, Song et al. [**?**] map to CSE-3 various compression operators in a pipeline. And isn't there a deep learning reference? https://training-api.cerebras.ai/en/rel-2.3.1/original/cerebras-basics/cerebras-execution-modes.html and https://dl.acm.org/doi/10.1145/3372780.3380846 how about these? GMG, however, is ill-suited to a pipelined layout. We must load all of initial grid onto the wafer to perform the computation. Pipelining operators requires storing and communicating intermediate data of the same size. Due to limited on-chip memory, the storage of intermediate data in the finest grid will limit the problem size that can be supported, and will introduce significant communication between operators and layers to reorganize the data.

b) Spatial Mapping of Multigrid Levels:: Suppose that different levels of the multigrid computation utilize adjacent portions of the wafer. As the finest grid performs the bulk of the computation, most of the wafer will hold the fine grid, and remaining PEs will mostly be idle. While under-utilization of idle PEs is a concern, the main flaw in this approach, similar to the pipelined strategy, is the significant communication of output/input data between levels at each level transition.

c) Distributed Computation Mapping with Spatial Level Mapping: The distributed mapping approach inspired by traditional MPI domain decomposition, superimposes layers on top of each other, such that each PE can perform all the operations. The finest grid level occupies the full region of the wafer. We call this spatial level mapping because data from coarser levels are either mapped towards one side or towards the center of the wafer. The stencils at each level can be highly parallelized given their nearest neighbor halo exchanges. This mapping across the whole wafer also avoids the under-utilization of the other solutions. This mapping aligns closely with WSE-3's wavelet-triggered dataflow execution model along with static color configurations to overlap communication and computations which can work across layers. Moreover, one can reuse stencil implementations such as [**?**], [**?**]. As with the spatial mapping above, the downside of this approach is the significant data communication and packing/unpacking costs as the GMG levels get coarser.

d) Strided Distributed Mapping (this paper): Multi-grids typically have more compute and data at the finer grids, so solving them fast is important. Also reducing the inter-layer cost is essential. The strided distributed mapping increases the distance between layers by powers of 2. Meaning a PE 2^l hops apart is the grid point for that level l. This reduces the inter-layer communication cost as compared to the two previous distributed mappings as the distance between layers increases. We describe in detail the strided distributed mapping used in this experiment in the remaining subsections.

B. Alternative approaches for level placement

Mapping different multigrid levels onto a spatial accelerator with thousands of independent PEs introduces an interesting design challenge. In addition to domain (i.e., data) partitioning, one has to also perform spatial partitioning and layout of the computation (operators and levels), and routing of data amongst levels. GMG's V-cycle poses unique challenges as the levels have irregular data sizes, and there are data dependences

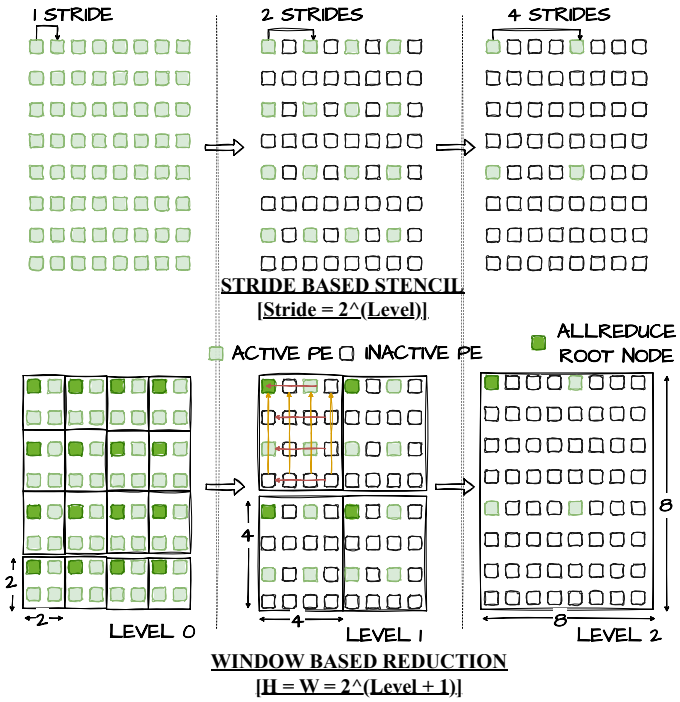

Fig. 3. Top: Stencil Operator on WSE-3 2D grid for a 8^3 problem with 3 levels. The Highlighted nodes are the Active PEs and remaining Inactive PEs. The Hop distance differs as one keeps going down the levels and the SpMV exchanges halo regions H hops apart

across levels in the iterative algorithm. All of these factors must be considered to arrive at the GMG mapping strategy.

In this subsection, we discuss different level placement strategies and their tradeoffs: fully distributed, spatial, and operator pipelined layouts, as shown in Figure **??**. Each approach has tradeoffs and implications for memory footprint, data communication distance, packing-repacking cost, parallelism, router and coloring code complexities, and amount of chip utilization and power consumption. These are captured in Table **??**. With the advent of large-scale spatial-dataflow accelerators, one can program the distributed machine as a single device with a single programming model. Therefore it amplifies the importance of placement as routing and communication is directly tied to layout and resulting performance.

1) Spatial Mapping: In the spatial layout shown in Figure **??**, each level of multigrid is spatially mapped either next to each other or far apart but connected via data paths. Each level uses a different set of PEs. This approach provides a low-per PE memory usage, and high locality for stencils as neighbors are 1 hop apart. But it suffers from high repacking and communication costs across levels. This mapping results in low overall utilization as many PEs remain idle while waiting for neighboring levels to complete.

2) Operator Pipelined Mapping: In operator pipelined layout, one can place layers all along the vertical dimension next to each other or even at a finer granularity to pipeline the layers along with their operators shown in the Figure **??**. The depth of the pipeline will be $\text{depth} = 2 * \log_2 N * iter_{\text{smooth}}$, where N is the size of one dimension of the input problem. The wavelets carry intermediate results from one level to the next and keep feeding the pipeline for the next iteration as they exit the other end of the chip. The closest related work that uses an operator pipelining is by Song et al. [**?**], where the authors map various compression operators in a pipeline. This strategy is useful for data parallel and feedforward kinds of workloads. GMG, however, has data dependences from intermediate data which creates a lot of communication overhead on the fabric when transitioning between layers. Some layers/operators can be idle waiting for data from other computationally-expensive layers/operators to finish causing pipeline bubbles and synchronization stalls. In addition, the PE parallelism decreases due to dividing layers along the horizontal dimension. The operator code must be duplicated, and the implementation of placement and routing might also become complex.

3) Distributed Mapping: centered-overlapped and one-sided overlapped: The distributed mapping approach inspired by traditional MPI domain decomposition, superimposes layers on top of each other, such that each PE can perform all the operations, as shown in the Figure **??**. The finest level occupies the full region of the wafer, while coarser levels are either mapped towards one side (top left) or towards the center (middle left). The stencils can be highly parallelized given their nearest neighbor halo exchanges. This mapping across the whole wafer also avoids the under-utilization of the other solutions. This mapping aligns closely with WSE-3's wavelet-triggered dataflow execution model along with static color

configurations to overlap communication and computations which can work across layers. Moreover, one can reuse stencil implementations such as [**?**], [**?**]. There can be significant data communication costs as the GMG layers get coarser.

4) Strided Distributed Mapping (this paper): Multi-grids typically have more compute and data at the finer grids, so solving them fast is important. Also reducing the inter-layer cost is essential. The strided distributed mapping increases the distance between layers by powers of 2. Meaning a PE 2^l hops apart is the grid point for that level l. This reduces the inter-layer cost as compared to the two previous distributed mappings as the distance between layers increases. We describe in detail the strided distributed mapping used in this experiment in the remaining subsections.

C. From 3D problem domain to 2D wafer

The GMG input domain is a structured 3D $nx \times ny \times nz$ grid as shown in Figure 1. We map a 3D problem onto a 2D grid by storing the entire z-dimension (nz elements) on each PE for a fixed value of x and y; i.e., a *pencil*. Given the 48KB memory on each PE, this decomposition enables us to scale the problem by using more PEs. All data accesses along the z dimension are local and do not require communication. This approach has been studied in previous stencil mappings on WSE [**?**], [**?**], [**?**], [**?**]. Given the multi-grid dimensions halve at each level, we have to maintain in memory multiple *pencils* from different levels as we superimpose levels. The total amount of data that will fit is a function of levels, L, and pencil size nz_l for each level $l \in L$. The total allocation along the z-axis per PE is a finite geometric series with $|L|$ levels and nz the size of the finest-level pencil l_0,

$$\text{Allocation} = nz + nz/2 + nz/4 + nz/8 + ...$$

$$\text{Allocation} = nz\left(2 - \frac{1}{2^{l-1}}\right)$$

Allocation represents the amount of data that must fit in 48KB memory along with other intermediate vectors and kernel buffers, and the program code for the PE. The finest levels in GMG need the highest amount of data and are computationally expensive; therefore, layer l_0 requires the most memory.

D. V-Cycle Execution as Tasks & State Machine

The separate operators of the V-cycle are treated as separate tasks and activated by wavelets as described in Section II-B3, corresponding to states in a state machine. To make the V-cycle discussion complete, we present the state machine transitions from the implementation in Table II, where each operator is shown with its computations at each state. We synchronize all PEs beforehand so that timers are aligned to the same start time.

The state machine iterates by advancing the states starting from the down cycle. It sets up the level, applies a Jacobi 7-pt stencil operator as a smooth, restricts by reducing residual values within a window to the top-left PE of each window and dividing by 8 (finite-volume). For each level from the coarsest back to the finest the coarse solution in the z-dimension is expanded by duplicating each z-slice to double the z-size. We broadcast the correction from the top-left PE of each restriction window to all PEs in that window, and add the interpolated correction. The key kernels of the algorithm are 7_pt_stencil, reduce_to_topleft, and broadcast_from_topleft. The latter two are inter-level communication kernels and the former is computed at the current level. Kernels are placed across all PEs of the grid; given the distributed interposing each level can reuse the kernels. I don't understand what this means, but I think it can be omitted: There is no special routing for levels as it is handled by the operators internally.

1. coloring 2. layout 3. tasks, data state

• The implementation of GMG on WSE-3 requires composing all of the programming model elements described in Section 2.2.3—layout, coloring, routing, tasks, DSDs, and runtime router reconfiguration—into a single coordinated program. The full implementation comprises approximately 8,900 lines of CSL device code and 1,800 lines of Python host code. Table I summarizes the code organization.

• Layout and coloring. The layout module $(\text{layout}_g mg_v cycle.csl, 157 lines) defines a 2D PE grid on the 762 1172 WSI$ $3 fabric. It allocates 9 colors for inter- PE communication | 8 for the stencil operator's directional halo exchang$ $3 for the stencil's send/receive/compute phases, 4 for all reduce stages, c$ $cycle state machine. These 17 hardware resources must be partitioned wi$ $dependent parameters (boundary flags, per- PE stencil routing) and binding them to a single kernel file via @set_tile_co$

• Routing and forwarding. Static routing is established at compile time: each PE's comptime block configures 16 $@\text{set}_l ocal_c olor_c on fig directives that wire input/output queues to the fou$ are 2^l hops apart, we modified the upstream SDK stencil and allreduce libraries (3,766 lines) to support runtime route reconfiguration. At each level transition, setHopBasedParams recomputes which PEs are active, which are interior forwarders, and adjusts the send/receive patterns accordingly. The forwarding optimization (Section 3.6) uses hardware queues and microthreads to relay wavelets through inactive PEs directly from input queue to output queue, bypassing the memory ramp entirely.

• Kernel and state machine. The V-cycle kernel $(\text{kernel}_g mg_v cycle.csl, 1, 103 lines) implements the full multigrid algorit$ $device state machine with 28 states. Each GM Goperator | smooth, residu$ $driven : when an asynchronous stencil or allreduce completes, the libra$ $cycle without returning control to the host between states; the host only in$ $cycle iterations to read convergence data.$

• Data management within 48KB. Each PE stores the solution vector u, right-hand side f, residual r, and stencil output Au for the current level, plus multi-level save arrays $(\text{save}_u smooth, save_f down) sized by the geometric series n_z(2-$ $2^{1-L})$. All data access uses DSDs (Data Structure Descriptors)—62 DSD operations configure strided memory access patterns for operations like restriction (even/odd z-pairs) and interpolation (element duplication). The stencil communication libraries add four directional buffers of $\text{BLOCK}_S IZE each for halo exchange, bringing per-$

TABLE I
CODE BREAKDOWN OF THE GLOW IMPLEMENTATION.

Component	Role	Lines
Device (CSL)		
kernel_gmg_vcycle.csl	State machine, operators	1,103
layout_gmg_vcycle.csl	PE grid, colors, exports	157
stencil_3d_7pts (hops)	Strided halo exchange	2,189
allreduce (hops)	Windowed reduction/bcast	1,577
blas.csl + timer	Local math, timestamps	309
Host (Python)		
run_gmg_vcycle.py	Orchestration, profiling	770
compile_and_run_wse3.py	Compilation, caching	554
util.py + cmd_parser.py	Data layout, arguments	489
Total		**7,148**

TABLE II
STATE MACHINE SUPPORTING SYNCHRONIZATION.

State name	Action
Smooth	
STATE_DOWN_INIT	set window and stride parameters
STATE_DOWN_SMOOTH_APPLY	$Ax_1 \leftarrow \texttt{stencil}(A, x)$
STATE_DOWN_SMOOTH_UPDATE	$x_{\text{smooth}} \leftarrow x + Ax_1$
Residual	
STATE_DOWN_RESIDUAL_APPLY	$Ax_2 \leftarrow \texttt{stencil}(A, x_{\text{smooth}})$
STATE_DOWN_RESIDUAL_COMPUTE	$r \leftarrow b - Ax_2$
Restriction	
STATE_DOWN_RESTRICT_ALLREDUCE	$b \leftarrow \texttt{reduce\_to\_topleft}(r)$
STATE_DOWN_RESTRICT_DIV	$b_{\text{next}} \leftarrow b/8$
Interpolation	
STATE_UP_INIT	set level, restore data
STATE_UP_EXPAND_Z	$x_{\text{coarse}} \leftarrow \texttt{expand}(x_{\text{fine}})$
STATE_UP_BCAST	$I \leftarrow \texttt{broadcast\_from\_topleft}($
Error Correction	
STATE_INTERP_ADD	$x \leftarrow x_{\text{smooth}} + I$
Coarse Solver	
STATE_COARSE_SOLVER_INIT	set level
STATE_COARSE_SOLVER_APPLY	$Ax\_coarse \leftarrow \texttt{stencil}(A, x\_coarse)$

$PE data pressure close to the 48KB limit for the largest problem sizes. At 512^3$ with 9 levels, the halo block size must be reduced from 256 to fit within memory alongside the program code, which itself occupies approximately 47

The key message this conveys to the reader: implementing a single multigrid V-cycle on WSE-3 requires the programmer to manually manage color/task ID allocation, static and dynamic routing, DSD-based memory access patterns, callback-driven asynchronous state machines, and per-PE memory budgeting within 48KB—all concerns that are abstracted away on conventional architectures. The 3,766-line modified communication library alone (forked from the SDK) reflects the engineering cost of adapting existing nearest-neighbor stencil code to the strided multi-level setting.

E. Performance Models of Key Operators

This subsection describes the internals of the key GMG operators in GLOW, that must be adapted to a distributed, tiled architecture. We discuss the compute- and communication-heavy operators (7-point stencil, and restriction) that mostly dominate the V-cycle cost. We use an analytical model to characterize the costs of operators in our implementation and how they affect the runtime.

a) Strided 7-point Stencil:: Mathias et al. present a low-latency 25-point stencil operator with localized broadcast patterns on WSE-2, achieving near peak performance [**?**]; the authors contributed a similar 7-point stencil kernel as part of upstream `sdk-examples`. Using this code as a starting point, we extend the communication across PEs to fit our strided mapping strategy.

Figure 3 illustrates how the stencil operator works on the WSE-3 for an 8^3 problem size with 3 levels. The green highlighted nodes are the active PEs for the current level, and the rest are inactive. By active, we mean that the PE is a participant in the stencil operation at that particular level and performs halo exchanges with neighbors. The inactive PEs are not participants in the stencil operation although they must participate in communication of data. The stride distance between active PEs changes with level and grows by 2^l in both x and y direction. The accumulator receives the neighboring halo data from 4 directions (E/W/N/S). Also, the Dirichlet boundary conditions on edges initialize the halos with negation of edges. Each PE, holds the stencil coefficients in SRAM and computes the local Laplacian operation across both x and y dimension and local z dimension, only if the PE is active. For communication, each PE sends data sequentially to all 4 cardinal directions and asynchronously receives. We observe that this mapping contributes to various costs like contention in network, distance of communication, amount of data over the network, cost of moving data to or from ramp and router.

To model these costs, we leverage the performance model of Luczynski et al. [**?**], which focuses on modeling collective communication operations. We model the inter- and intra-communication costs, and router to memory access cost with respect to the level of the GMG for the strided distributed mapping strategy.

Per Luczynski et al., Equation (1) estimates the number of cycles T_l at level l for a given problem size. The model has two terms. In the first term, C is the largest number of wavelets a PE sends or receives, which is a measure of contention. The second term captures the routing cost: E is the total number of hops the network needs to route wavelets; N is the number of links being used (a PE has 5 bidirectional links); and, L is the largest number of hops a wavelet has to travel. Thus, the first term reflects whether contention or routing the communication pattern is dominant. In the second term, T_R is the router-processor latency (multiplied by 2 for sending and receiving data and an additional cycle for storing the received element),

TABLE III
COMMUNICATION MODEL (EQ. 2) FOR STRIDED MAPPING: $C_l = \texttt{zDIM}_0/2^l$, $L_l = D_l = 2^l$, $N_l = 4$; $\texttt{zDIM}_0 = 512$, $T_R = 2$. COST BREAKDOWN: *Latency* = $(2T_R{+}1)D_l = 5D_l$.

l	**Grid**	C_l	L_l	D_l	**Comm. cost**	**Latency cost**	**Total** T_l
0	512×512	512	1	1	512	5	517
1	256×256	256	2	2	256	10	266
2	128×128	128	4	4	128	20	148
3	64×64	64	8	8	64	40	104
4	32×32	32	16	16	32	80	112
5	16×16	16	32	32	36	160	196
6	8×8	8	64	64	68	320	388
7	4×4	4	128	128	132	640	772
8	2×2	2	256	256	260	1280	1540

and D is the longest pipeline sequence of PEs that perform operations depending on previous calculations. Combining the two terms provides a model for communication costs, compute and memory access costs.

$$T_l = \max\left(C_l, E_l/N_l + L_l\right) + (2T_R + 1)D_l \tag{1}$$

Next, we apply this model for each level of a 16^3 sized problem for three levels, as shown in Figure 3, with varying number of hops at each level. The data that must be communicated is the pencil resident at each PE and $C_l = C_{l0}/2^l$, where $C_{l0} = zDim_0$ and the number of hops is $L_l = 2^l$. The number of dependent PEs, D_l, is equal to distance between active PEs, as the stencil pattern sends to only one neighbor in each direction,$N_l = 4$. Hence, we conclude

$$T_l = \max\left(zDim_0/2^l, C_l * L_l/N_l + L_l\right) + (2T_R + 1)D_l \tag{2}$$

Therefore, for the context of this paper, the data that must be communicated is the pencil resident at each PE for a given level, and the number of hops it must be communicated is a function of the level. For our *strided mapping*, the distance between active PEs at level l is $D_l = 2^l$, and the pencil size is $C_l = \texttt{zDim}_0/2^l$. Then, $L_l = 2^l$ hops and $N_l = 4$ directions (stencil sends to one neighbor per direction). Table III instantiates these terms for a 512^3 problem across 9 levels.

b) Window-based Restriction:: For the restriction operator, we adopt a window-based approach at each level as well. We adapt the allreduce primitive from the upstream `sdk-examples`, extending it to operate on hierarchical windows of PEs. At each GMG level. The reduction across points to coarsen the grid is performed within a window of equal height and width, which is a function of the level. Let W be window size at level l, then $W = 2^{(l+1)}$. The window reduction pattern is shown in Figure 3, where the dark green PE on top left is the receiver. Within each window, the algorithm performs a row-wise reduction followed by a column-wise reduction across the window width and height. Due to the geometry of the layout, each window contains exactly two active PEs per direction.

The cost of restriction, based on Equation (1), is as follows. $C_l = \texttt{zDim}_0/2^l$, $W = 2^{(l+1)}$, $L_l = D_l = W + W$ (Manhattan Distance to farthest PEs), $N_l = 1$, $T_R = 2$. Clearly, W dominates the cost and the total time to restrict the data from the farthest PE to the top-left PE. Performance observations indicate that short-distance communication achieves significant wins, whereas longer-distance communication incurs higher cost, primarily due to memory-access overheads associated with reductions involving inactive PEs.

F. Forwarding Optimization

Table III exposes a key bottleneck in long-distance communication. We observe that as the data reduces per PE and distance increases, the communication cost term in our model decreases initially and starts increasing again along with the latency cost growing by a factor of 2. In the strided mapping, the latency term

$$LatencyCost = (2T_R + 1)D_l = 5D_l \tag{3}$$

dominates the communication term from level 5 (D_l) in Table III; it equals the number of pipeline stages (hops) between active processing elements (PEs). This growth occurs because each hop requires access to the ramp and local memory, which can transfer only one wavelet at a time. As a result, wavelets traveling across multiple hops must repeatedly pass through memory, causing serialization in the pipeline and increasing latency.

Table IV demonstrates that we can substantially reduce this communication and latency if we bound the number of memory accesses. This optimization is possible in the strided approach as we can skip the memory access on inactive PEs, leading to a $D\_l = 2$ for sender and receiver accesses. Using this optimization, we can significantly speedup the time. We achieve this reduction using hardware resources – microthreads and queues – so that the incoming wavelets are forwarded directly to the output queue without intermediate memory accesses. This bypasses the ramp and effectively reduces the costly access so that the latency no longer grows with level. We compare our optimized implementation in Section IV-E.

IV. EVALUATION

In this section, we evaluate the performance of GMG implemented on the Cerebras WSE-3 and compare it against the state-of-the-art HPGMG (High Performance Geometric Multigrid) GPU benchmark.

A. Experimental Configuration

We used the CS-3 cluster in the Argonne National Laboratory Compute Facility (ALCF), which consists of 4 WSE-3 machines along with proprietary memory nodes called MemoryX. One WSE-3 wafer integrates 893,064 processing elements (PEs) arranged in an 1172×762 2D mesh operating at 875 MHz along with 44 GBs of total on-chip SRAM

TABLE IV
COMMUNICATION MODEL WITH **CONSTANT LATENCY**: $C_l = \texttt{zDIM}_0/2^l$, $L_l = 2^l$, $N_l = 4$; $\mathbf{D}_l = \mathbf{2}$; $\texttt{zDIM}_0 = 512$, $T_R = 2$. COST BREAKDOWN: *Latency* = $5D_l = 10$.

l	**Grid**	C_l	L_l	D_l	**Comm. cost**	**Latency cost**	**Total** T_l
0	512×512	512	1	2	512	10	522
1	256×256	256	2	2	256	10	266
2	128×128	128	4	2	128	10	138
3	64×64	64	8	2	64	10	74
4	32×32	32	16	2	32	10	42
5	16×16	16	32	2	36	10	46
6	8×8	8	64	2	68	10	78
7	4×4	4	128	2	132	10	142
8	2×2	2	256	2	260	10	270

memory. We used Cerebras SDK[1] version 1.4.0. The experiments were conducted on a single WSE-3 attached to a host system running Linux kernel 4.18.0 on an AMD EPYC 9354P 32-Core Processor with 188 GB of main memory. For the GPU baseline, we used an NVIDIA Grace Hopper GH200 at the TACC HPC facility[2], equipped with 120 GB RAM and running CUDA 12.5(g). To ensure compatibility, we modified the baseline HPGMG code[3] to use single precision data, as WSE-3 does not support double precision.

The CSL backend compiler was invoked with the LLVM options `--inline-threshold=256` and `--unroll-threshold=256`. Experiments on the GH200 system were conducted with the following modules loaded: `openmpi/5.0.5`, `ucc/1.5.1`, `ucx/1.19.1`, `cmake/4.1.1`, `xalt/3.1`, `TACC`, `nvidia/24.7`, `nvpl/24.7`.

B. Problem Description

We evaluate the performance of GMG implemented entirely on the device. All experiments solve the constant-coefficient 3D Poisson equation on a structured Cartesian grid using a 7-point 3D-stencil. The domain is discretized uniformly in all three dimensions, and homogeneous Dirichlet boundary conditions are imposed on the domain boundaries. The grid spacing of h^2 is uniform in all three dimensions.

The right hand side of Ax=b is initialized to:

$$b = \sin(\pi x)\sin(\pi y)\sin(\pi z)$$

where x, y and z are spatial locations. The initial solution x initialized to 10.0, and the stencil coefficients are: $\alpha = -6.0$, $\beta = 1.0$. Jacobi relaxation with $\omega = 2/3$ is used as the smoother. The coarsest level goes to a domain size of 2x2, except for shallow V-cycle experiments. Convergence is determined based on the maximum norm of the residual at the finest level, and solver iterations terminate once a prescribed tolerance still true?: ($\epsilon = 10^{-5}$) is reached or a maximum number of V-cycles is reached.

All timing measurements are collected on-device using hardware counters and excludes host-to-device and device-to-host transfers. We report total time per V-cycle as well as a breakdown of time spent in various operators. All experiments are performed in single precision (FP32) arithmetic. We show various experiments that change the number of pre-smoothing and post-smoothing and coarse-smoothing iterations at the coarsest level. Unless otherwise stated we show the experiments with 6/6/6 (pre-/post-/coarse) as the default configuration as that turns out to be the best-performing configuration for GLOW. The hardware uses 16 channels, and up to a 256 block size for the halo exchange buffers. (Note that due to the 48KB memory limit for both code and data, along with storing data for multiple levels at a node, the halo exchange for the 512^3 problem size must reduce its halo exchange granularity due to memory constraints (see Section IV-E).

C. V-Cycle Variation Performance Comparison

Figure 4 reports the performance of GLOW on WSE-3 compared against HPGMG on NVIDIA GH200. Bars above the speedup of 1 baseline bar mean WSE-3 is faster. The x-axis denotes the 2D PE grid size under weak scaling; the z-dimension of the problem maps one-to-one onto the grid, so larger grids correspond to larger 3D problems. Each bar in a group corresponds to a distinct multigrid configuration experiment, parameterized by the number of pre-, post-, and coarse-level smoothing iterations. GLOW consistently outperforms HPGMG on the GH200, with speedups increasing as problem size scales. The best speedup, approximately $25\times$, is observed at the 512×512 grid for the $6/6/6(shallow)$ configuration. In contrast, configurations with aggressive coarse-level smoothing, such as $4/4/100$, show the lowest speedups, particularly at larger grid sizes of 256×256.

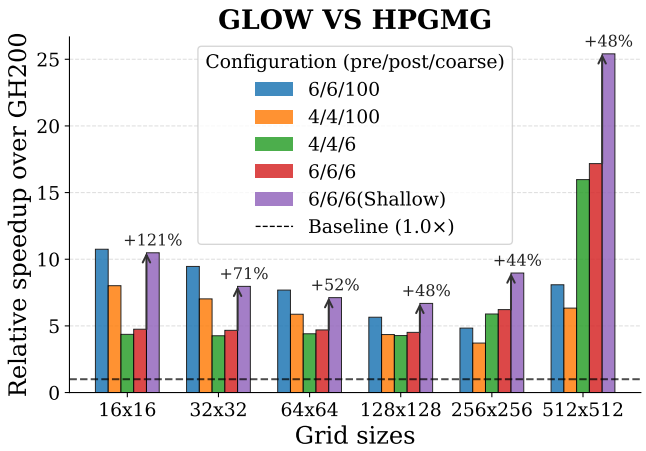

Fig. 4. Speedup comparison between Cerebras CS-3 and NVIDIA GH200 using HPGMG/Glow across various problem sizes. The bar plot shows relative performance (higher is better). The % increase indicated on top of 6/6/6(Shallow)(purple) is the relative speedup against the 6/6/6(red),

These performance trends highlight the sensitivity of GMG performance on WSE-3 to V-cycle structure. Increasing the

[1]https://sdk.cerebras.net/installation-guide
[2]https://docs.tacc.utexas.edu/hpc/vista/
[3]https://hpgmg.org/

number of coarse-level iterations (*/*/6 vs. */*/100) along with depth of the V-cycle expands communication distances for both inter-level transfers and intra-level stencil operations. In contrast, our implementation handles finer grids well by scaling the problem over the wafer and exploiting parallelism and stencil locality, with shorter communication distances for halo exchanges and fewer hops to travel for inter-layer communication. A $6/6/6(Shallow)$ V-cycle with 7 levels outperforms a $6/6/6(Deep)$ V-cycle with 9 levels by almost 45% on the largest problem size, with the cost of increased iterations to reach convergence.

As a second observation, speedups grow as problem size scales. GLOW is well suited to finer levels and larger grids, while single-node GPU performance worsens on larger problems due to increased data movement through the memory system.

The configurations with a heavy number of coarse solver iterations (i.e., the blue and orange bars) exhibit a U-shaped trend in Figure 4, as the cost of the coarse solver dominates for larger problems, pushing the performance to lower levels. Configurations with lighter coarse solver iterations (green and red bars) show a linear increase in performance as the problem scales since very less time is spent on the coarse level. A higher bump on the 512 problem size could also point to how baseline GPUs start struggling with larger grids and not GLOW performing better by $2\times$ over previous level.

D. Per-Operator Performance Breakdown

We attribute the time spent to individual operators in Figure 5. The plot shows time per operator for one V-cycle on a 512^3 problem, summed over each horizontal level (both down and up phases combined). The x-axis is the multi-grid level from 0 (finest) to 8 (coarsest); the y-axis is execution time in a log scale (lower is faster).

Smooth with 7-point (strided) stencil. At levels above L_1, a strided stencil refers to skipping PEs in the calculation as a result of the grid coarsening. As is typical in GMG, smooth time initially decreases toward coarser levels because both the number of active PEs and the local data volume are halved at each level. However, unlike conventional weak-scaling behavior, where performance often plateaus/saturates or improves monotonically toward coarser grids, in GLOW we observe a gradual increase in cost beyond a certain level.

To better understand this behavior, we microbenchmarked the strided stencil kernel used at the heart of the smooth and residual operators, as shown in Figure 6. We measured compute time to calculate the Laplacian across the x-y plane and z-axis. Communication time is the time taken to send and receive halo regions to and from neighbors from all 4 directions. Compute time decreases nearly by a factor of two per level before flattening, due to fixed overheads. Communication cost starts lower than compute on finer grids as the stencil halo neighbors are exchanged directly between neighbors, while the data size is largest and has maximal computation. The intersection point of these curves reflects the tradeoff between shrinking data volume and increasing communication distance: although each PE processes less data, messages must traverse longer paths as the active region contracts.

We compare analytical predictions in III and Table IV with these performance measurements. One can observe the curves with and without forwarding-optimization both show a higher communication cost in the initial finer levels and latency grows as we move towards coarser levels. There is a sweet spot where the communication cost and latency just overlap each other. The following sentence may cause reviewers to say that we should have done this already. Also, I think it may belong in a subsequent "Lessons Learned" section: To further reduce the cost, we believe even more sophisticated techniques to change the routing might be useful, one such is demonstrated by Mathias et al. [**?**] using control wavelets and runtime coloring reconfiguration.

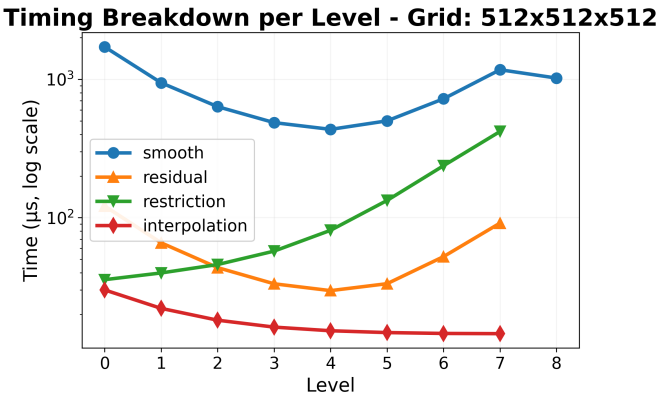

Fig. 5. Per-operator time per level for one V-cycle on a 512^3 problem (GLOW on CS-3). Down and up phases are summed at each level. x-axis: level 0 (finest) to 8 (coarsest). y-axis: time (log scale; higher is slower). Config: $4/4/6$ pre/post/coarse, 9 levels, FP32, 7-point stencil.

Residual. The residual operator uses the same 7-point stencil as smoothing and follows a similar performance trend. Differences in absolute time come from the absence of multiple smoothing iterations.

Restriction. Restriction exhibits increasing cost toward coarser levels. The window-based reduction (Figure 3) technique used in our distributed mapping requires contributions from $N-1$ other PEs within a window, introducing dependence and repeated accesses to local memory via the ramp. As the window size grows relative to the number of active PEs, these dependences dominate execution time. Skipping inactive or out-of-domain PEs during the reduction could potentially reduce this overhead and can be a promising direction for future optimization.

Interpolation: Interpolation shows the opposite behavior compared to restriction; it is more expensive on finer grids and becomes progressively cheaper toward coarser levels. As microbenchmarked in Figure 7, interpolation cost is dominated by sending data compared to computing locally. Although one might expect the cost to increase with distance, instead interpolation benefits from the hardware's zero-cost multicast capability. Data is broadcast in all four directions simultane-

ously, allowing row-wise and column-wise transfers to occur in parallel. Unlike restriction, interpolation lacks producer-consumer dependences between PEs, enabling immediate forwarding without intermediate memory accesses. Consequently, each PE can multicast and advance, thus avoiding the high cost of going down the ramp to memory. As the data volume shrinks at coarser levels, smaller packets are sent over longer distances, but the use of multicast and the avoidance of ramp traffic result in lower overall cost. In Figure 7, we see that at each level the window-leader PE (top-left, as shown in Figure 3) expands the z dimension, resets the routes, broadcasts data, and eventually when data reaches the destination, calculates the error correction locally using tightly coupled DSD operations. Notice the cost to reset routes is constant mostly.

Coarse solver. The coarse solver applies weighted Jacobi relaxation on the coarsest grid. Its cost is very high compared to fine-level operators and grows with the number of coarse iterations. We showed experiments in Figure 4 with varying coarse iterations, significantly impacting overall performance in deep V-cycles.

E. Forwarding Communication Optimization

We further quantify the impact of the forwarding optimization of Section III-F. in Figure 8. The forwarding optimization shines as the number of inactive PEs start increasing. We see almost 8.45× speedup over the unoptimized/inactive PE memory accessing implementation. This leads to a 3.94× speedup over the whole application shown in the bottom left corner. Compute time reduces since unnecessary computation on inactive PEs is also eliminated.

F. Memory Constraints: Code vs. Data Tradeoff

The bottom-right panel of Figure 10 shows the breakdown of memory usage within a single processing element (PE), comparing the total available local memory against the size of the binary ELF image loaded on each PE. As shown, the code section alone occupies nearly 47% of the available 48 KB SRAM and remains constant across multigrid levels. This is largely driven by aggressive function inlining performed by the backend LLVM compiler within the state-machine implementation, as well as the inclusion of stencil and collective-reduction library kernels. In contrast, the data section scales with problem size and the number of multigrid levels, reflecting the storage required for solution vectors, residuals, and intermediate buffers. Under the distributed superimposed mapping strategy adopted in this work, these memory constraints impose a practical limit on scalability. In particular, the fixed code footprint combined with per-level data storage restricts the ability to reduce per-PE data volume, effectively limiting experiments to weak scaling. Moreover, multigrid algorithms inherently require retaining data from finer levels during traversal, further constraining memory reuse under a distributed mapping.

You should revisit the 512-cubed case needing a different implementation.

This should be in a Lessons Learned section, probably: These observations highlight an important design trade-off for wafer-scale implementations for multilayered algorithms/applications. Future approaches may benefit from hybrid mapping strategies that decouple computation and storage, such as assigning subsets of PEs primarily as memory resources while others focus on computation, or employing more dynamic coloring and routing schemes to reduce code replication and improve memory utilization across levels.

V. Lessons Learned and Next Steps

Grabbed from above: SOME INTRO PARAGRAPH...

a) Further improvements to forwarding optimization.: To further reduce the cost, we believe even more sophisticated techniques to change the routing might be useful, one such is demonstrated by Mathias et al. [**?**] using control wavelets and runtime coloring reconfiguration.

b) Addressing constrained memory size.: Revisit 512-cubed...Future approaches may benefit from hybrid mapping strategies that decouple computation and storage, such as assigning subsets of PEs primarily as memory resources while others focus on computation, or employing more dynamic coloring and routing schemes to reduce code replication and improve memory utilization across levels.

1. Various mapping (???), localized based on application as much as possible, memory constraints. 2. Conventional distributed mappings, 3. Pipeline for this application doesn't make sense, partition problems, distance 4. Programming model(async and hangs), colorings(design sophscticated patterns) 5. Mppaing, memory, distance of communication, convergence(HPC low radix machines for larger distances and cerebras needs to think about it), uniform communication patterns. 6.

We chose to investigate only the V-cycle in this paper, because it is by far the most common multigrid schedule in practice and because our per-level performance analysis lets the reader translate our findings directly to F-, W-, or μ-cycle variants. From a computational-efficiency standpoint, all parallel architectures favour the V-cycle: it hides coarse-grid latency and overheads behind the throughput-limited, bandwidth-bound fine grids. On the WSE, however, this effect is far more pronounced because coarse-grid operations pay a communication cost that grows with level rather than shrinking: the active-PE stride doubles per level, the inter-PE hop distance grows with it, and the network is low-radix, so each SpMV/reduction/broadcast at level ℓ is increasingly latency-dominated. On a 256^3 problem implemented using W cycle with 8 levels, and ($\gamma = 2$) we see the time per 1 cycle increases from 483.1 useconds(V-cycle) to 15,103.9 useconds(W-cycle), which is almost a 31x slowdown for W cycle. W-cycle visits level L exactly 2^L times, accounting for the penalty.

VI. Related Work

Historically, operations on large structured grids could easily be bound by capacity misses in cache, leading to a variety of studies on blocking and tiling optimizations to

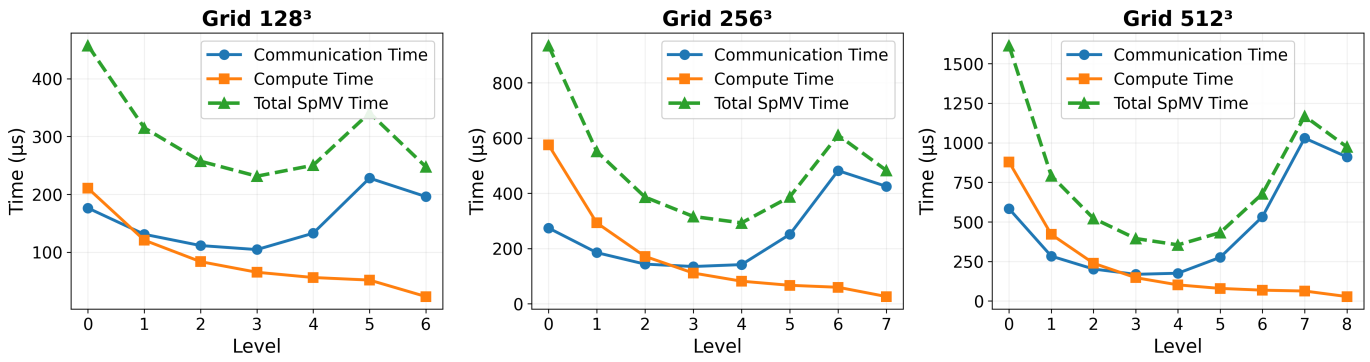

Fig. 6. Internal speedup of smooth on Cerebras CS-3 and NVIDIA GH200 using HPGMG and GLOW across various problem sizes. The bar plot shows relative performance (higher is better).

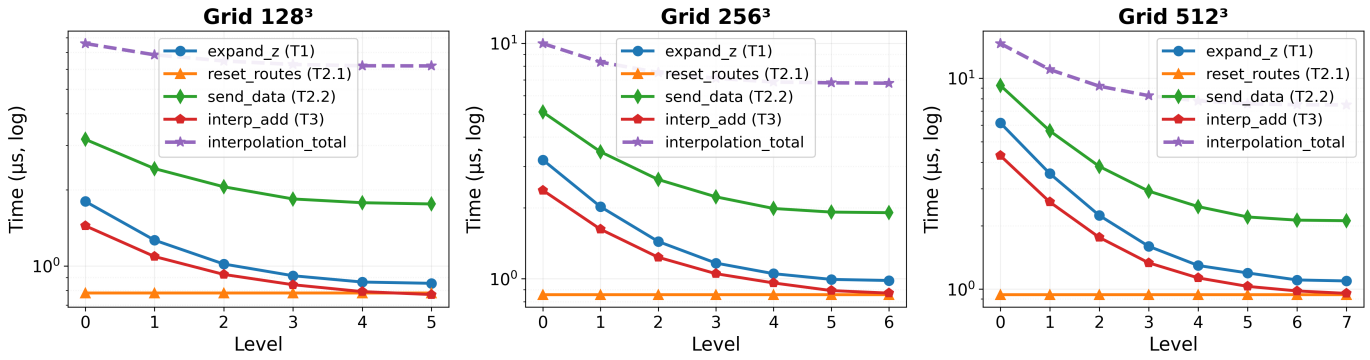

Fig. 7. Internal speedup of interpolation on Cerebras CS-3 and NVIDIA GH200 using HPGMG/Glow across various problem sizes. The bar plot shows relative performance (higher is better).

improve cache reuse on CPUs [?], [?], [?], [?], [?]. These techniques for multigrid have been successfully applied to finite-difference and finite-volume discretizations on shared-memory and distributed-memory systems. As new supercomputer architectures evolve with higher peak multi-node performance, the growing gap between DRAM bandwidth and floating point operations (FLOPs) has focused research on increasing performance through data locality, particularly for bandwidth-bound computations.

Other efforts have introduced MG frameworks that could efficiently exploit resources on GPU architectures. Examples of recent multiphysics applications, where multigrid solvers are key for elliptic equations, on structured or unstructured grids include [?], [?], [?], [?]. Many of these efforts have been focused on GPU acceleration and, to some extent performance portability, since the focus is on different supercomputers with different GPU vendors. However, because of the complexity of multigrid solvers, it is an ongoing challenge to do detailed performance analysis and explore optimization techniques that exploit increasing GPU parallelism, GPU memory, and network bandwidth.

A growing body of work has demonstrated the effectiveness of wafer-scale systems for scientific computing. Early studies showed near-ideal weak scaling for high-order stencil computations and structured finite-difference operators on the WSE [?], [?], [?], [?], [?]. Subsequent work extended these ideas to matrix-free finite-volume solvers for subsurface flow and seismic applications, highlighting the advantages of spatial data placement and explicit communication control [?], [?]. Larger application studies include molecular dynamics simulations that overcome traditional timescale barriers [?], [?] and Monte Carlo particle transport kernels achieving substantial speedups over GPU-based implementations [?], [?]. Communication-intensive primitives such as reductions and all-reductions have also been studied in isolation, with near-optimal collective implementations demonstrated on wafer-scale fabrics [?].

In parallel, several efforts have focused on improving programmability and compiler support for wafer-scale systems. These include DSL-based approaches for stencil computation [?], automated code generation for high-order stencils [?], and MLIR-based lowering pipelines targeting spatial dataflow

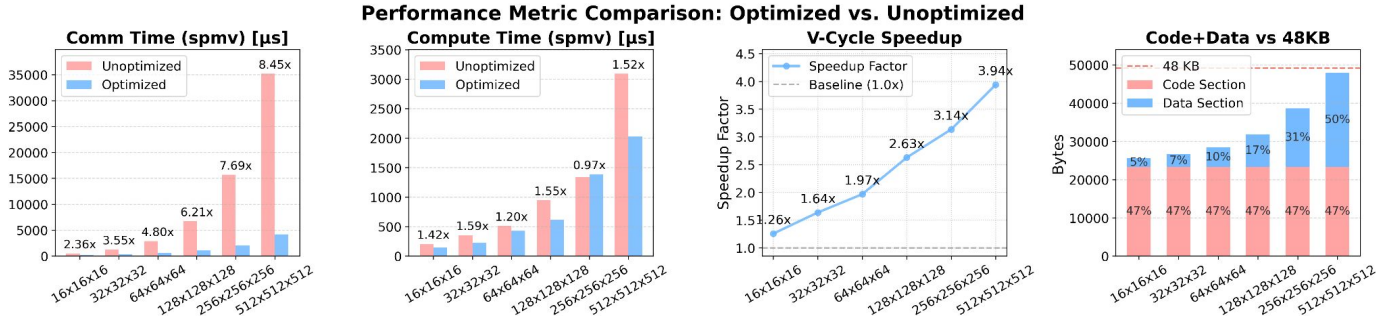

Fig. 8. Performance of HPGMG/Glow on Cerebras CS-3 and NVIDIA GH200 using a 512^3 grid with 9 multigrid levels, up to 100 iterations, a convergence tolerance of 1×10^{-5}, $4/4/6$ pre-, post-, and bottom-smoothing steps, float32 precision, Jacobi relaxation $\omega = 0.6667$, a 7-point stencil with $\alpha = -6.0$, $\beta = 1.0$, 16 channels, 256 block size, and a 762×1172 wafer fabric.

architectures [**?**]. SPADA [**?**] is a programming language targeting spatial architectures that provides precise control over data placement, dataflow patterns, and asynchronous operations while abstracting architecture-specific low-level details. Various literature on analytically modeling and spatial algorithmic primitives have been explored as well [**?**], [**?**], [**?**], [**?**]. While these works significantly advance kernel-level performance and developer productivity, they primarily target single-level stencil operators or application-specific kernels.

To the best of our knowledge, no prior work has demonstrated a complete geometric multigrid solver mapped onto a wafer-scale architecture. GMG introduces additional challenges beyond single-level stencils, including multi-level data grids, coarse-grid solvers, and level-dependent communication patterns. This work represents the first end-to-end exploration of mapping a full GMG algorithm onto the Cerebras WSE-3 system, bridging the gap between prior stencil-centric studies and full multi-level scientific solvers.

VII. Conclusion

We presented a mapping of GMG to WSE-3, achieving up to a $25\times$ speedup over a single node NVIDIA H200 GPU. As this is, to our knowledge, the first mapping of GMG to WSE-3, this work exposes the challenges of achieving efficiency for layered algorithms, especially when memory size is so constrained. As a result of this work, we make a number of observations regarding mapping multigrid algorithms to tiled architectures. First, the compute time dominates for the fine grids, but communication time dominates and increases execution time for coarser grids due to the strided communication. Therefore, shallow versus deep V-cycles or W-cycles are better suited for this architecture. The communication pattern required to skip over inactive PEs was optimized with a forwarding optimization that lead to significant speedups for coarse grids. In spite of these challenges, deterministic, high-bandwidth memory access times and local network bandwidth lead to speedups over a single GPU.

***Index Terms*—Spatial dataflow architecture, Geometric multigrid, Wafer-Scale Engine**

References

Please number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use "Ref. [3]" or "reference [3]" except at the beginning of a sentence: "Reference [3] was the first ..."

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors' names; do not use "et al.". Papers that have not been published, even if they have been submitted for publication, should be cited as "unpublished" [4]. Papers that have been accepted for publication should be cited as "in press" [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

References

[1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.

[2] J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

[3] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

[4] K. Elissa, "Title of paper if known," unpublished.

[5] R. Nicole, "Title of paper with only first word capitalized," J. Name Stand. Abbrev., in press.

[6] Y. Yorozu, M. Hirano, K. Oka, and Y. Tagawa, "Electron spectroscopy studies on magneto-optical media and plastic substrate interface," IEEE Transl. J. Magn. Japan, vol. 2, pp. 740–741, August 1987 [Digests 9th Annual Conf. Magnetics Japan, p. 301, 1982].

[7] M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template

text from your paper may result in your paper not being published.